

Data Science, VL 3

Dr. Konrad Völkel, Sommersemester 2026

23.04.2025

Einführung in Python

- REPL ist unser Freund

Erste Schritte in der REPL (Video) unter media.hhu.de/channel/video/data-science-python-in-der-repl/1383cd1ccc36f42479354b1c8c0dd373/167

Einführung in Python

Heinrich Heine
Universität Düsseldorf

- REPL ist unser Freund
- Einrückung statt Klammern

Erste Schritte in der REPL (Video) unter media.hhu.de/channel/video/data-science-python-in-der-repl/1383cd1ccc36f42479354b1c8c0dd373/167

Einführung in Python

Heinrich Heine
Universität Düsseldorf

- REPL ist unser Freund
- Einrückung statt Klammern
- Alles ist ein Objekt

Erste Schritte in der REPL (Video) unter media.hhu.de/channel/video/data-science-python-in-der-repl/1383cd1ccc36f42479354b1c8c0dd373/167

Einführung in Python

Heinrich Heine
Universität Düsseldorf

- REPL ist unser Freund
- Einrückung statt Klammern
- Alles ist ein Objekt
- LLMs können Python

Erste Schritte in der REPL (Video) unter media.hhu.de/channel/video/data-science-python-in-der-repl/1383cd1ccc36f42479354b1c8c0dd373/167

Zahlen: `int` und `float`

- `int`: Ganze Zahlen, die beliebig groß werden können (kein Overflow).

Zahlen: `int` und `float`

- `int`: Ganze Zahlen, die beliebig groß werden können (kein Overflow).
- `float`: Kommazahlen (IEEE 754 binary64 double precision auf den meisten Plattformen).

Zahlen: `int` und `float`

- `int`: Ganze Zahlen, die beliebig groß werden können (kein Overflow).
- `float`: Kommazahlen (IEEE 754 binary64 double precision auf den meisten Plattformen).
- Typen müssen nicht explizit deklariert werden, Python bindet Variablen dynamisch.

Zahlen: `int` und `float`

- `int`: Ganze Zahlen, die beliebig groß werden können (kein Overflow).
- `float`: Kommazahlen (IEEE 754 binary64 double precision auf den meisten Plattformen).
- Typen müssen nicht explizit deklariert werden, Python bindet Variablen dynamisch.
- Operatoren: `+`, `-`, `*`, `/`.

Zahlen: `int` und `float`

Heinrich Heine
Universität Düsseldorf

- `int`: Ganze Zahlen, die beliebig groß werden können (kein Overflow).
- `float`: Kommazahlen (IEEE 754 binary64 double precision auf den meisten Plattformen).
- Typen müssen nicht explizit deklariert werden, Python bindet Variablen dynamisch.
- Operatoren: `+`, `-`, `*`, `/`.
- Besonderheiten: `//` für ganzzahlige Division und `%` für Modulo.

Text: Strings (`str`)

- `str` repräsentiert Text als Sequenzen von Unicode-Zeichen.

Text: Strings (`str`)

Heinrich Heine
Universität Düsseldorf

- `str` repräsentiert Text als Sequenzen von Unicode-Zeichen.
- Strings werden mit einfachen oder doppelten Anführungszeichen erstellt.

Text: Strings (`str`)

- `str` repräsentiert Text als Sequenzen von Unicode-Zeichen.
- Strings werden mit einfachen oder doppelten Anführungszeichen erstellt.
- Strings sind unveränderlich (immutable); Modifikationen erzeugen neue Objekte.

Text: Strings (`str`)

- `str` repräsentiert Text als Sequenzen von Unicode-Zeichen.
- Strings werden mit einfachen oder doppelten Anführungszeichen erstellt.
- Strings sind unveränderlich (immutable); Modifikationen erzeugen neue Objekte.
- Zugriff auf einzelne Zeichen oder Teilbereiche erfolgt über Slicing (`x[1:4]`).

Wahrheitswerte und `None`

- `bool`: Der Datentyp für Wahrheitswerte mit den Konstruktoren `True` und `False`.

Wahrheitswerte und `None`

- `bool`: Der Datentyp für Wahrheitswerte mit den Konstruktoren `True` und `False`.
- Nahezu jedes Objekt kann in Python zu einem Boolean evaluiert werden (z.B. leere Sammlungen sind `False`).

Wahrheitswerte und `None`

- `bool`: Der Datentyp für Wahrheitswerte mit den Konstruktoren `True` und `False`.
- Nahezu jedes Objekt kann in Python zu einem Boolean evaluiert werden (z.B. leere Sammlungen sind `False`).
- `None`: Ein spezielles Objekt (null in Java).

Wahrheitswerte und `None`

- `bool`: Der Datentyp für Wahrheitswerte mit den Konstruktoren `True` und `False`.
- Nahezu jedes Objekt kann in Python zu einem Boolean evaluiert werden (z.B. leere Sammlungen sind `False`).
- `None`: Ein spezielles Objekt (null in Java).
- Nützlich, um Variablen zu deklarieren, die noch keinen Wert haben.

Listen (`list`)

- Eine geordnete, veränderbare und dynamische Sammlung von Objekten (ähnlich ArrayList).

Listen (`list`)

- Eine geordnete, veränderbare und dynamische Sammlung von Objekten (ähnlich ArrayList).
- Werden mit eckigen Klammern `[]` oder `list()` initialisiert.

Listen (`list`)

- Eine geordnete, veränderbare und dynamische Sammlung von Objekten (ähnlich ArrayList).
- Werden mit eckigen Klammern `[]` oder `list()` initialisiert.
- Längenabfrage funktioniert einfach über die eingebaute Funktion `len(liste)`.

Dictionaries (`dict`)

- Speichern von Key-Value-Paaren, (eine Hash-Map).

Dictionaries (`dict`)

- Speichern von Key-Value-Paaren, (eine Hash-Map).
- Die Syntax nutzt geschweifte Klammern und Doppelpunkte: `{"key": "value"}`.

Dictionaries (`dict`)

- Speichern von Key-Value-Paaren, (eine Hash-Map).
- Die Syntax nutzt geschweifte Klammern und Doppelpunkte: `{"key": "value"}`.
- Als Keys (Schlüssel) sind nur unveränderliche Typen wie `str`, `int` oder `float` erlaubt.

Dictionaries (`dict`)

Heinrich Heine
Universität Düsseldorf

- Speichern von Key-Value-Paaren, (eine Hash-Map).
- Die Syntax nutzt geschweifte Klammern und Doppelpunkte: `{"key": "value"}`.
- Als Keys (Schlüssel) sind nur unveränderliche Typen wie `str`, `int` oder `float` erlaubt.
- Wenn ein angefragter Key nicht existiert, wirft Python einen `KeyError`.

Fallunterscheidungen (`if` / `else`)

- Python verzichtet auf Klammern zur Abgrenzung von Codeblöcken.

Fallunterscheidungen (`if` / `else`)

- Python verzichtet auf Klammern zur Abgrenzung von Codeblöcken.
- Blöcke werden durch einen Doppelpunkt eingeleitet und eingerückt (4 Leerzeichen sind 'ein Tab').

Schleifen (`while` und `for`)

Heinrich Heine
Universität Düsseldorf

- `while`-Schleifen prüfen eine Bedingung und wiederholen den Körper, bis diese nicht mehr zutrifft.

Schleifen (`while` und `for`)

Heinrich Heine
Universität Düsseldorf

- `while`-Schleifen prüfen eine Bedingung und wiederholen den Körper, bis diese nicht mehr zutrifft.
- `for`-Schleifen in Python sind eigentlich for-each-Schleifen: Sie iterieren direkt über Sammlungen wie Listen oder Dictionaries.

Schleifen (`while` und `for`)

Heinrich Heine
Universität Düsseldorf

- `while`-Schleifen prüfen eine Bedingung und wiederholen den Körper, bis diese nicht mehr zutrifft.
- `for`-Schleifen in Python sind eigentlich for-each-Schleifen: Sie iterieren direkt über Sammlungen wie Listen oder Dictionaries.
- Um klassisch über Zahlen zu iterieren, nutzt man das Objekt `range(a, b)`.

Funktionen definieren (`def`)

- Funktionen werden mit dem Schlüsselwort `def` eingeleitet.

Funktionen definieren (`def`)

- Funktionen werden mit dem Schlüsselwort `def` eingeleitet.
- Im Gegensatz zu `if` oder Schleifen erzeugt `def` einen eigenen Geltungsbereich (Scope) für Variablen.

Funktionen definieren (`def`)

- Funktionen werden mit dem Schlüsselwort `def` eingeleitet.
- Im Gegensatz zu `if` oder Schleifen erzeugt `def` einen eigenen Geltungsbereich (Scope) für Variablen.
- Funktionen können optionale Argumente (mit Default-Werten) und Keyword-Argumente annehmen.



Heinrich Heine
Universität Düsseldorf

Vielen Dank für Ihre Aufmerksamkeit!